



React



Agenda

- Introducción
- Create React App (deprecado 14 Febrero 2025), usaremos Vite
- JSX
- Componentes
- Propiedades
- Estado
- Renderizado Condicional
- Renderizado de Elementos
- Eventos
- Comunicación entre componentes
- Ciclo de Vida
- Hooks

¿Qué es React?

React es una **librería** Javascript altamente eficiente y declarativa que se utiliza para crear interfaces de usuario interactivas.

React te incita a crear código declarativo, y orientado a **componentes**. Nace con la programación reactiva.

Renderiza usando el Virtual DOM.

Ventajas

- Alto rendimiento
- Recursos abundantes
- Compatibilidad con versiones anteriores
- Estructura de componentes fácil de mantener
- Fuerte comunidad
- Documentación multi idioma
- Flujo de datos unidireccional

Desventajas

- Complejo
- JSX
- Necesidad de un ecosistema de muchas herramientas
- Problemas de SEO

Entorno y Herramientas de Desarrollo

- Node.js y NPM
- Yarn
- Webpack
- React router
- Redux
- Create react app (deprecado 2025, usaremos Vite)
- Next.js
- React bootstrap
- Material UI
- etc.

Create React App (deprecado) -> Vite

Instalar:

- Node y NPM

Correr comando:

- `npm create vite@latest`

Que involucra create react app:

- Webpack, babel, PostCSS, Jest, etc.

Comandos:

- `npm install`, `npm run dev`, `npm run build`.

Sintaxis JSX

```
<div className="App">
  <header className="App-header">
    <img src={logo} className="App-logo" alt="logo" />
    <p>
      Edit <code>src/App.js</code> and save to reload.
    </p>
    <a
      className="App-link"
      href="https://reactjs.org"
      target="_blank"
      rel="noopener noreferrer"
    >
      Learn React
    </a>
  </header>
</div>
```

Componentes

- Un componente es un elemento autónomo que se reutiliza en una página.
- Pueden tener estados (variables internas) y propiedades (**props**).
- La comunicación es de componente padre a componente hijo.
- REACT te permite definir componentes como clases o como funciones.

Como una clase que extiende de **Component** con un método **render**.

```
import React, { Component } from "react";

class Title extends Component {
  render() {
    return <h1>Hola mundo</h1>;
  }
}
```

O como una función que retorna lo que se va a renderizar:

```
const Title = () => {
  return <h1>Hola Mundo</h1>;
};
```

Propiedades

- Son valores que recibe un componente hijo de uno padre. Se agrupan en un objeto llamado props, el cual puede recibir diferentes tipos de datos, como:
 - Strings
 - Numbers
 - Booleans
 - Arrays
 - Objects
 - Functions
 - React Elements
 - React Components
- Las props son inmutables, es decir, son valores de sólo lectura, no se pueden modificar.
- El componente las recibe como atributos que se pasan a través de JSX.

Estado

El state son los valores internos que manejan la lógica y los datos de un componente, permite que éste reaccione a cualquier cambio lo que hará que se vuelva a renderizar en la interfaz.

El estado tiene 3 características importantes:

- Es inmutable.
- No se puede modificar directamente.
- Es asíncrono.

```
class Welcome extends Component {  
  constructor() {  
    super();  
  
    this.state = {  
      title: "Hola Mundo",  
    };  
  }  
  
  render() {  
    return <h1>{this.state.title}</h1>;  
  }  
}
```

Práctica

- Instalar “Nodejs” (chequear si no está instalado con `node -v`)
- Instalar “Visual Studio Code”
- Correr el comando “`npm create vite@latest`”, elegir un nombre para el proyecto, como framework elegir “React” y como lenguaje “Javascript”. Moverse a la carpeta creada y correr “`npm install`”.
- Crear carpeta Componente
 - dentro de la carpeta “Componente”, agregar un archivo extensión .jsx y crear un componente de clase y otro funcional al cual le pueda pasar una prop, mostrar el valor enviado en pantalla.

Eventos y Binding

```
class Welcome extends Component {
  constructor() {
    super();

    this.state = {
      title: "Hola Mundo",
    };

    // tenemos que enlazar el método al contexto actual
    this.updateText = this.updateText.bind(this);
  }

  updateText() {
    this.setState({
      title: "Hello World",
    });
  }

  render() {
    return <h1 onClick={this.updateText}>{this.state.title}</h1>;
  }
}
```

Comunicación entre componentes

Tenemos 3 casos de comunicación entre los componentes de *React*:

1. Comunicación entre un componente padre a uno hijo.
2. Comunicación entre un componente hijo y su padre.
3. Comunicación entre componentes no relacionados.

Ciclo de Vida

- Montaje
 - constructor()
 - render()
 - componentDidMount()
 - ver ajax y apis
- Actualización
 - render()
 - componentDidUpdate()
- Desmontaje
 - componentWillUnmount()

Ajax Apis

AJAX no es un lenguaje de programación

AJAX = Asynchronous JavaScript and XML (JavaScript asíncrono y XML).

Es una técnica de desarrollo web para crear aplicaciones interactivas

Podemos usar:

- XHR (XMLHttpRequest)
- Fetch
- Promises

Hooks

Los *Hooks* son una nueva incorporación a partir de **React 16.8.0**, que nos permiten "**enganchar**" el estado y el ciclo de vida en componentes basados en funciones.

¿Por qué se crearon los *Hooks*?

Las clases confunden a las personas y a las máquinas.

Entender la estructura y reglas que implican crear una clase puede ser una curva de aprendizaje lenta y requerir de ciertos conceptos, como el manejo de *this* en *JavaScript*, por el contrario las funciones son muy fáciles de entender y manipular incluso para personas que van comenzando.

Tipos de Hooks

- Básicos (en el 100% de tus proyectos):
 - ***useState.***
 - ***useEffect.***
- Avanzados (tal vez no en todos tus proyectos):
 - ***useContext.***
 - ***useRef.***
 - ***useReducer.***
 - ***useCallback.***
 - ***useMemo.***

Hook useState

```
import React, { useState } from "react";

export default function Componente() {
  const [valor, setValor] = useState(0);
  return (
    <div>
      <span>El valor del componente es {valor}</span>
      <button onClick={() => setValor(valor + 1)}>Aumentar valor</button>
    </div>
  );
}
```

Hook useEffect

```
export default function ScrollyNavegador() {  
  const [scrollY, setScrollY] = useState(0);  
  
  useEffect(() => {  
    //Creamos una función para actualizar el estado  
    const actualizarScrollY = () => {  
      let scrollY = window.pageYOffset;  
      console.log(`scrollY: ${scrollY}`);  
      setScrollY(scrollY);  
    };  
    //Actualizamos el scroll al montar el componente  
    actualizarScrollY();  
    //Nos suscribimos al evento scroll de window  
    window.addEventListener("scroll", actualizarScrollY);  
  
    //Devolvemos una función para desuscribir el evento  
    return () => {  
      window.removeEventListener("scroll", actualizarScrollY);  
    };  
  });  
  
  return (  
    <div>  
      <span>ScrollY del Navegador: {scrollY}px</span>  
    </div>  
  );  
}
```

Hooks Personalizados

Un Hook personalizado es una función de JavaScript cuyo nombre comienza con "use" y que puede llamar a otros Hooks.

```
import { useState, useEffect } from 'react';

function useFriendStatus(friendID) {
  const [isOnline, setIsOnline] = useState(null);

  useEffect(() => {
    function handleStatusChange(status) {
      setIsOnline(status.isOnline);
    }

    ChatAPI.subscribeToFriendStatus(friendID, handleStatusChange);
    return () => {
      ChatAPI.unsubscribeFromFriendStatus(friendID, handleStatusChange);
    };
  });

  return isOnline;
}
```

Referencias y el DOM

Las referencias proporcionan una forma de acceder a los nodos del DOM o a elementos React creados en el método de renderizado.

```
class MyComponent extends React.Component {  
  constructor(props) {  
    super(props);  
    this.myRef = React.createRef();  
  }  
  render() {  
    return <div ref={this.myRef} />;  
  }  
}
```

Hooks useRef

```
function TextInputWithFocusButton() {  
  const inputEl = useRef(null);  
  const onClick = () => {  
    // `current` points to the mounted text input element  
    inputEl.current.focus();  
  };  
  return (  
    <>  
      <input ref={inputEl} type="text" />  
      <button onClick={onClick}>Focus the input</button>  
    </>  
  );  
}
```

Componentes controlados y no controlados

Un componente controlado es aquel que usa los cambios de estado o cambios de `props` como fuente de verdad para representarse en el `DOM`.

Un componente no controlado, es aquel que no usa el estado o las `props` para representarse en el `DOM`, y, por el contrario, usa la [API del DOM](#). La manera en la que React obtiene los valores es usando la [API de REF](#).

Los **formularios** en React son buenos casos de uso para aplicar ambos patrones.

Estilos CSS

- Importando un archivo
- En línea
- Trayendo normalize
- Como módulos
- Utilizando sass (npm install node-sass)

Frameworks CSS con React

- Bootstrap
- Bulma

React Bootstrap & Material UI

Bootstrap

It is an HTML, CSS & JS framework to make responsive websites that are mobile-friendly and easy to create.

Material UI

Material UI is a highly interactive & customizable framework based on React UI and Material Design.

CRUD App

1. Creación de componentes y renderizado de datos
2. Inserción de datos
3. Edición de datos y comunicación entre componentes
4. Eliminación de datos y estilos CSS